Lecture 1.6: Loops — `for`, `while`, `do...while`

Why loops?

A loop repeats a block of code until a condition becomes false. Loops help us avoid writing the same code again and again.

`for` loop

A `for` loop is best when you know how many times you want to repeat.

```
// Print 0 to 9
for (var i = 0; i < 10; i++) {
    console.log(i);
}
```

Parts of a for loop

- **Initialization:** `var i = 0`
- **Condition:** `i < 10` (loop runs while this is true)
- **Update:** `i++` (after each iteration)

`while` loop

A `while` loop is best when you want to repeat until some condition changes, and you might not know the exact number of repeats.

```
// Print 0 to 10
var i = 0;
while (i <= 10) {
```


Powered by SPRK Technologies

```
    console.log(i);
    i++;
}
```

> **Common bug:** If you forget `i++`, the loop may become an **infinite loop**.

`do...while` loop

A `do...while` loop runs the body **at least once**, even if the condition is false at the beginning.

```
// Print 1 to 10
var i = 1;
do {
    console.log(i);
    i++;
} while (i <= 10);
```

When to use which loop?

Loop	Best for	Key property
`for`	Counting (fixed or known range)	All loop parts are together
`while`	Unknown number of iterations	Condition checked before body
`do...while`	Must run at least once	Condition checked after body

Task: multiplication table

Task Solution: Ask the user for a number and print its multiplication table.

Powered by SPRK Technologies

Solution using a loop

```
var nText = prompt("Enter a number for the table:");

if (nText !== null) {
    var n = parseInt(nText, 10);

    if (isNaN(n)) {
        console.error("Invalid number");
    } else {
        for (var i = 1; i <= 10; i++) {
            console.log(`${n} x ${i} = ${n * i}`);
        }
    }
}
```

Output:

For `5`:

`5 x 1 = 5`

`5 x 2 = 10`

...

`5 x 10 = 50`

Bonus: `break` and `continue`

Both `break` and `continue` change the normal flow of a loop. They are useful, but they can also make code harder to follow if overused.

`break`: stop the loop completely

Use `break` when you have found what you are looking for, or when continuing is unnecessary. When `break` runs, the loop ends immediately and execution continues *after* the loop.

Powered by SPRK Technologies

```
// Example 1 (for loop): find the first multiple of 7
for (var i = 1; i <= 50; i++) {
    if (i % 7 === 0) {
        console.log("First multiple of 7 is:", i);
        break; // stop as soon as we found it
    }
}
```

Output:

```
First multiple of 7 is: 7
```

```
// Example 2 (while loop): keep asking until the user types "exit"
// (Pseudo demo: in real code, prompt runs in the browser)
while (true) {
    var text = prompt("Type something (or 'exit' to stop):");
    if (text === null || text.toLowerCase() === "exit") {
        break; // ends the loop
    }
    console.log("You typed:", text);
}

console.log("Loop ended");
```

> **Tip:** `while (true)` with a `break` is a common pattern for "repeat until we decide to stop". Just make sure the `break` condition is reachable.

`continue`: skip this iteration, go to the next

Use `continue` when you want to ignore certain cases, but still keep looping. When `continue` runs, the current iteration ends immediately and the loop starts the next one.

```
// Example: print 1..10 but skip 5
for (var i = 1; i <= 10; i++) {
    if (i === 5) {
        continue; // skip printing 5
```


Powered by SPRK Technologies

```
    }
    console.log(i);
}
```

Output:

```
1
2
3
4
6
7
8
9
10
```

```
// Example: print only odd numbers between 1 and 10
for (var i = 1; i <= 10; i++) {
    if (i % 2 === 0) {
        continue; // skip even numbers
    }
    console.log(i);
}
```

Output:

```
1
3
5
7
9
```


Powered by SPRK Technologies

Important note: `break` / `continue` in nested loops

In nested loops, a plain `break` or `continue` affects only the **closest** (inner) loop.

```
// Nested loops: break only exits the inner loop
for (var row = 1; row <= 3; row++) {
    for (var col = 1; col <= 3; col++) {
        if (col === 2) {
            break;
        }
        console.log("row", row, "col", col);
    }
}
```

Output:

```
row 1 col 1
row 2 col 1
row 3 col 1
```

> **Optional advanced idea:** JavaScript also supports *labeled* `break` to exit an outer loop. This is rare in everyday code, but it exists.

JS Refined Notes — Lecture 1.6